Multi-Layer Perceptrons and Deep Learning

Christos Dimitrakakis

October 22, 2026

Outline

Features and layers

- Introduction
- Layers
- Activation functions

Algorithms

- Random projection
- Back propagation
- Derivatives
- Cost functions
- Gradient descent
- Stochastic gradient descent in practice
- Other learning rules

Python libraries

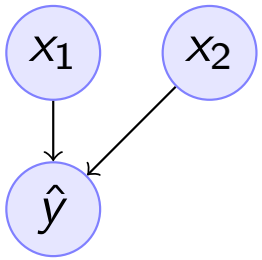
- sklearn
- PyTorch
- TensorFlow

Features and layers
- Introduction
- Layers
- Activation functions

Algorithms
- Random projection
- Back propagation
- Derivatives
- Cost functions
- Gradient descent
- Stochastic gradient descent in practice
- Other learning rules

Python libraries
- sklearn
- PyTorch
- TensorFlow

Perceptron vs linear regression

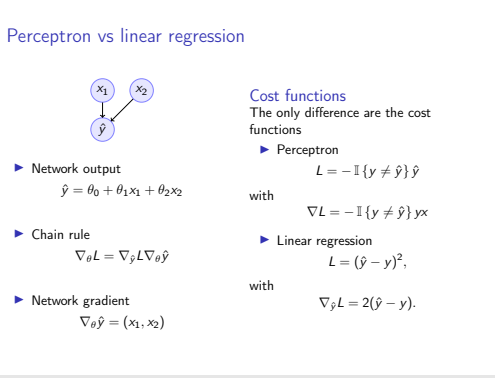


- Network output
$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2$$
- Chain rule
$$\nabla_\theta L = \nabla_{\hat{y}} L \nabla_\theta \hat{y}$$
- Network gradient
$$\nabla_\theta \hat{y} = (x_1, x_2)$$

Cost functions

The only difference are the cost functions

- Perceptron
$$L = -\mathbb{I}\{y \neq \hat{y}\}\hat{y}$$
with
$$\nabla L = -\mathbb{I}\{y \neq \hat{y}\} yx$$
- Linear regression
$$L = (\hat{y} - y)^2,$$
with
$$\nabla_{\hat{y}} L = 2(\hat{y} - y).$$



Summary. This example looks at a model with just two inputs, and three parameters. The graph determines dependencies. More specifically, the nodes are variables, and the arcs show that we need x_1, x_2 to calculate $\hat{y}$. We can use such graphical models as computation graphs or decision networks. A graphical model between variable $x_1, x_2, \cdots, x_n$, is a directed graph, saying which variables are direct inputs to computing other variables. Examples are given later.

Perceptron, linear regression and MLPs.

Both the perceptron and linear regression share the same structure. However, the variable $\hat{y}$ is interpreted differently. In the perceptron it is the distance (and side) from the hyperplane. In linear regression, it is the predicted target. For that reason, it makes sense to use a different function for the two methds.

We can obtain parameters for both models using the chain rule, to split the gradient of the loss with respect to the parameters in two parts: the gradient of the loss with respect to the output, and the gradient of the output with respect to the parameters. This principle can be extended to more complex transformations between x, y as well as other cost functions. A typical idea is to use a sequence of nonlinear transformations to project the data into higher or lower dimensional spaces. These are called layers.

Perceptron vs linear regression



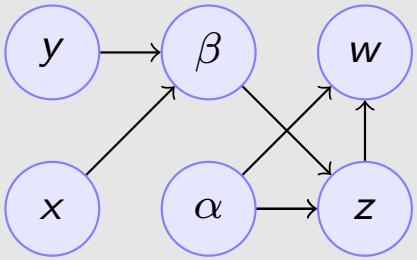
- Network output
  $\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2$
- Chain rule
  $\nabla_\theta L = \nabla_{\hat{y}} L \nabla_\theta \hat{y}$
- Network gradient
  $\nabla_\theta \hat{y} = (x_1, x_2)$

Cost functions
The only difference are the cost functions

- Perceptron
  $L = -\mathbb{I}\{y \neq \hat{y}\}\hat{y}$
  with
  $\nabla L = -\mathbb{I}\{y \neq \hat{y}\}yx$
- Linear regression
  $L = (\hat{y} - y)^2,$
  with
  $\nabla_{\hat{y}} L = 2(\hat{y} - y).$

Computation graphs. Let us start with a mathematical example:

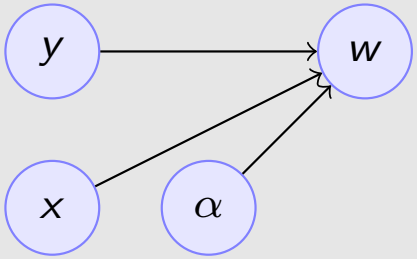
$$\beta = x + y, \qquad z = \alpha\beta, \qquad w = \exp(\alpha + z).$$



Here, w depends on two variables, and α influences two other variables. If we replace everything we get

$$w = \exp[\alpha + \alpha(x + y)] = \exp[\alpha(x + y + 1)].$$

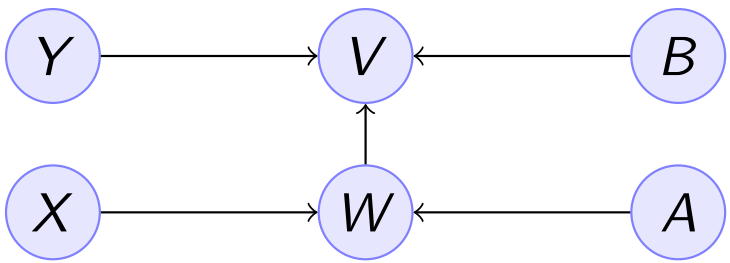
Now the intermediate variables are missing, so the graph becomes simpler. However, we need the intermediate variables if we want to compute something about them specifically (like their partial derivative).



Test

Let us now consider a python example

```
W = np.tanh(np.matmul(X, A))
V = np.tanh(np.matmul(W+Y, B))
```



Layering and features

Fixed layers

- Input to layer $x \in R^n$
- Output from layer $\hat{y} \in R^m$.

Intermediate layers

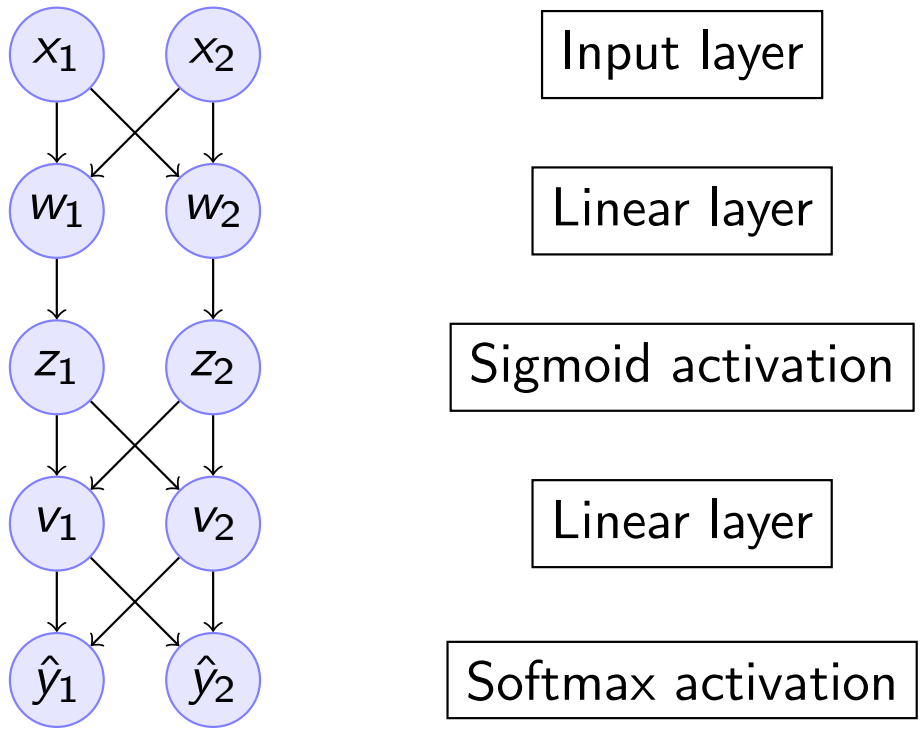
- Linear layer
- Non-linear activation function.

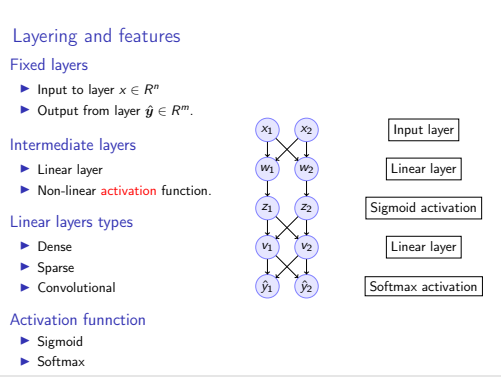
Linear layers types

- Dense
- Sparse
- Convolutional

Activation funnction

- Sigmoid
- Softmax





Explanation. Here we have a "neural network" with three "layers". The "input layer" is simply the observations. The first linear layer is called "hidden", and the last linear layer is called the "output" layer. Each intermediate output is called a "neuron".
For historical reasons related to neuroscience, the non-linear layers are called "activation" functions, because of how a neuron activates depending on the total incoming current.
The simplest type of layer is the linear, but here we have many types. A dense layer from n to m is multiplication with an $n \times m$ matrix. A sparse layer is a sparse such matrix. Convolutional layers are used in computer vision as a way to obtain space invariance.

Note that the softmax layer and the sigmoid layer have different dependencies. As you'll see later, the sigmoid layer involves an activation function for a single "neuron", while the softmax is like a "winner-takes-all" approach.

Linear layers

Example: Linear regression with n inputs, m outputs.

- Input: Features $\boldsymbol{x} \in \mathbb{R}^n$
- Dense linear layer with $\boldsymbol{\Theta} \in \mathbb{R}^{m \times n}$
- Output: $\hat{\boldsymbol{y}} \in \mathbb{R}^m$

Dense linear layer

- Parameters $\boldsymbol{\Theta} = \begin{pmatrix} \boldsymbol{\theta}_1 \\ \vdots \\ \boldsymbol{\theta}_m \end{pmatrix}$,
- $\boldsymbol{\theta}_i = [\theta_{i,1}, \ldots, \theta_{i,n}]$, $\boldsymbol{\theta}_i$ connects the i-th output y_i to the features $\boldsymbol{x}$:

$$y_i = \boldsymbol{\theta}_i \boldsymbol{x}$$

- In compact form:

$$\boldsymbol{y} = \boldsymbol{\Theta} \boldsymbol{x}$$

Linear layers

Example: Linear regression with n inputs, m outputs.

- Input: Features $\boldsymbol{x} \in \mathbb{R}^n$
- Dense linear layer with $\boldsymbol{\Theta} \in \mathbb{R}^{m \times n}$
- Output: $\hat{\boldsymbol{y}} \in \mathbb{R}^m$

Dense linear layer

- Parameters $\boldsymbol{\Theta} = \begin{pmatrix} \boldsymbol{\theta}_1 \\ \vdots \\ \boldsymbol{\theta}_m \end{pmatrix}$,
- $\boldsymbol{\theta}_i = [\theta_{i,1}, \ldots, \theta_{i,n}]$, $\boldsymbol{\theta}_i$ connects the i-th output y_i to the features $\boldsymbol{x}$:
$$y_i = \boldsymbol{\theta}_i \boldsymbol{x}$$
- In compact form:
$$\boldsymbol{y} = \boldsymbol{\Theta} \boldsymbol{x}$$

Sparse layers and convolution.

- A sparse linear layer is again $\boldsymbol{y} = \boldsymbol{\Theta} \boldsymbol{x}$, but $\boldsymbol{\Theta}$ is a sparse matrix (i.e. with a number of non-zero values in the order of $o(\sqrt{nm})$). Sparse matrices matrices are particularly useful for random projections.
- A convolutional layer is a special case of a linear layer with tied variables. There, the same parameter value is applied in many matrix positions. This is used to e.g. take into account spatial invariance in images. It can be thought of as convolving a small matrix with the whole image, rather a single bix matrix being applied everywhere.

Multiple linear layers

Repeated linear transformations are linear

It does not really help to have multiple linear layers one after the other. For example, if we transform $x \in \mathbb{R}^n$ to $z \in \mathbb{R}^k$ to $y \in \mathbb{R}^m$ through two matrices

$$z = Ax, \qquad A \in \mathbb{R}^{k \times n} \tag{1}$$

$$y = Bz, \qquad B \in \mathbb{R}^{m \times k} \tag{2}$$

We can rewrite y as

$$y = B(Ax) = (BA)x = Cx, \qquad C \in \mathbb{R}^{m \times n} \tag{3}$$

where $C = BA$.

- Successive linear layers have no advantage normally.[1]
- However, we can interlace them with non-linear activation functions.

[1] Multi-task learning might be an exception.

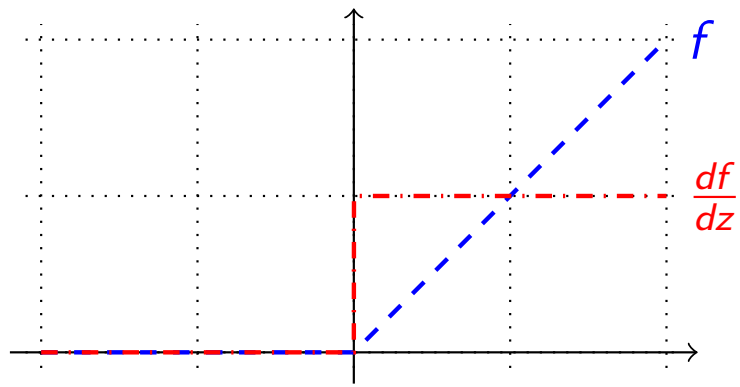
ReLU activation

- Activation function:

$$f(x) = \max(0, x)$$

- Derivative

$$\frac{d}{dx} f(x) = \mathbb{I}\{x > 0\}$$



Typically used in the hidden layers of neural networks, as it is:

- Simple to calculate.
- Nonlinear.
- Its gradient never vanishes.

Sigmoid activation

Example: Logistic regression

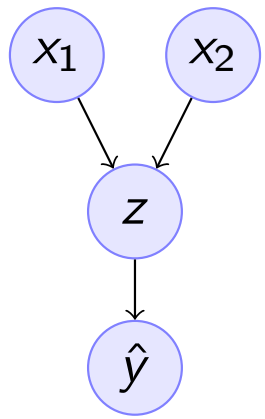
- Input $\boldsymbol{x} \in \mathbb{R}^n$, Labels $y \in \{0, 1\}$.
- Intermediate output: $z \in \mathbb{R}$,

$$z = \sum_{i=1}^{n} \theta_i x_i.$$

- Output: sigmoid activation $\hat{y} \in [0, 1]$.

$$f(z) = 1/[1 + \exp(-z)].$$

Now we can interpret $\hat{y} = P_{\boldsymbol{\theta}}(y = 1|x)$.



Input layer

Linear layer

Sigmoid layer

Loss function: negative log likelihood

$$\ell(\hat{y}, y) = -[\mathbb{I}\{y = 1\} \ln(\hat{y}) + \mathbb{I}\{y = 0\} \ln(1 - \hat{y})]$$

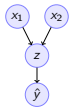
Sigmoid activation
Example: Logistic regression
- Input $x \in \mathbb{R}^n$, Labels $y \in \{0, 1\}$.
- Intermediate output: $z \in \mathbb{R}$,

$$z = \sum_{i=1}^{n} \theta_i x_i.$$

- Output: sigmoid activation $\hat{y} \in [0, 1]$.

$$f(z) = 1/[1 + \exp(-z)].$$

Now we can interpret $\hat{y} = P_\theta(y = 1|x)$.



Input layer
Linear layer
Sigmoid layer

Loss function: negative log likelihood

$$\ell(\hat{y}, y) = -[\mathbb{I}\{y = 1\} \ln(\hat{y}) + \mathbb{I}\{y = 0\} \ln(1 - \hat{y})]$$

When to use. Sigmoid activation has two main uses.

- For logistic regression, it is used as the output layer. Then the output corresponds to the probability of the positive class. In that case, it is combined with the negative log-likelihood loss. We can rewrite it as

$$\ell(\hat{y}, y) = \ln(\hat{y})^y + \ln(1 - \hat{y})^{1-y}.$$

- Sigmoid activations are also used in hidden layers, to confer some nonlinearity. However, these days ReLU activation is favoured as it is simpler and has a non-vanishing gradient.

Softmax layer

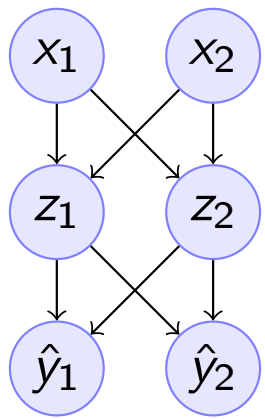
Example: Multivariate logistic regression with m classes.

- Input: Features $\boldsymbol{x} \in \mathbb{R}^n$
- Fully-connected linear activation layer

$$\boldsymbol{z} = \boldsymbol{\Theta}\boldsymbol{x}, \qquad \boldsymbol{\Theta} \in \mathbb{R}^{m \times n}.$$

- Softmax output

$$\hat{y}_i = \frac{\exp(z_i)}{\sum_{j=1}^{m} \exp(z_j)}$$



Input layer

Linear layer

Softmax layer

We can also interpret this as

$$\hat{y}_i \triangleq \mathbb{P}(y = i \mid \boldsymbol{x})$$

with usual loss $\ell(\hat{y}, y) = -\ln \hat{y}_y$

Softmax layer

Example: Multivariate logistic regression with m classes.

- Input: Features $\boldsymbol{x} \in \mathbb{R}^n$
- Fully-connected linear activation layer

$$\boldsymbol{z} = \boldsymbol{\Theta}\boldsymbol{x}, \qquad \boldsymbol{\Theta} \in \mathbb{R}^{m \times n}.$$

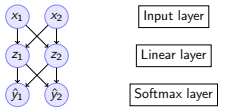
- Softmax output

$$\hat{y}_i = \frac{\exp(z_i)}{\sum_{j=1}^{m} \exp(z_j)}$$

We can also interpret this as

$$\hat{y}_i \triangleq \mathbb{P}(y = i \mid \boldsymbol{x})$$

with usual loss $\ell(\hat{y}, y) = -\ln \hat{y}_y$



Notes. The softmax layer can be seen as a soft *winner-takes-all* layer. For that reason, it is represented through connections from the previous layer to all the nodes. The higher the inputs are, the closer it is to winner-takes-all. In fact, sometimes you can add a factor β to represent this explicitly

$$\hat{y}_i = \frac{\exp(\beta z_i)}{\sum_{j=1}^{m} \exp(\beta z_j)}$$

When $\beta \to \infty$, the maximum output approaches 1 and the rest 0. When $\beta \to 0$, it approaches 0. However, β is usually simply set to 1.

- The standard use is an output layer in multi-class classification.
- It can also be used in intermediate layers, to obtain neurons that activate for specific types of data, while others are suppressed. It used in that way as part of attention mechanisms.

Features and layers
- Introduction
- Layers
- Activation functions

Algorithms
- Random projection
- Back propagation
- Derivatives
- Cost functions
- Gradient descent
- Stochastic gradient descent in practice
- Other learning rules

Python libraries
- sklearn
- PyTorch
- TensorFlow

Random projections

- Features x
- Hidden layer activation z
- Output y

Hidden layer: Random projection

Here we project the input into a high-dimensional space

$$z_i = \text{sgn}(\boldsymbol{\theta}_i^\top x) = y_i$$

where $\boldsymbol{\Theta} = [\boldsymbol{\theta}_i]_{i=1}^m$, $\theta_{i,j} \sim \text{Normal}(0, 1)$

The reason for random projections

- The high dimension makes it easier to learn.
- The randomness ensures we are not learning something spurious.

Random projections

- Features x
- Hidden layer activation z
- Output y

Hidden layer: Random projection

Here we project the input into a high-dimensional space

$$z_i = \text{sgn}(\boldsymbol{\theta}_i^\top x) = y_i$$

where $\boldsymbol{\Theta} = [\boldsymbol{\theta}_i]_{i=1}^m$, $\theta_{i,j} \sim \text{Normal}(0, 1)$

The reason for random projections

- The high dimension makes it easier to learn.
- The randomness ensures we are not learning something spurious.

Sparseness. While Θ is not sparse in the traditional sense, most of its values are close to zero, few are very large. Hence, it is a bit like a noisy sparse matrix.

Random projections are used either to reduce, or to increase dimensionality. If used to reduce dimensionality, it can be shown that distances between vectors are preserved with high probability.

Background on back-propagation

Gradient descent algorithm

- We need to minimise the expected value $\mathbb{E}_{\boldsymbol{\theta}}[L]$ of the loss function L
- Since we cannot calculate $\mathbb{E}_{\boldsymbol{\theta}}[L]$, we use:

$$\nabla_{\boldsymbol{\theta}} \mathbb{E}_{\boldsymbol{\theta}}[L] \approx \frac{1}{T} \sum_{t=1}^{T} \nabla_{\boldsymbol{\theta}} \ell(x^t, y^t, \boldsymbol{\theta}).$$

- We can then update our parameters to reduce the empirical loss

$$\boldsymbol{\theta}^{t+1} = \boldsymbol{\theta}^t - \alpha_t \nabla_{\boldsymbol{\theta}} \ell(x^t, y^t, \boldsymbol{\theta}).$$

The problem

- However ℓ is a complex function of $\boldsymbol{\theta}$.
- How can we obtain $\nabla_{\boldsymbol{\theta}} \ell$?

The solution

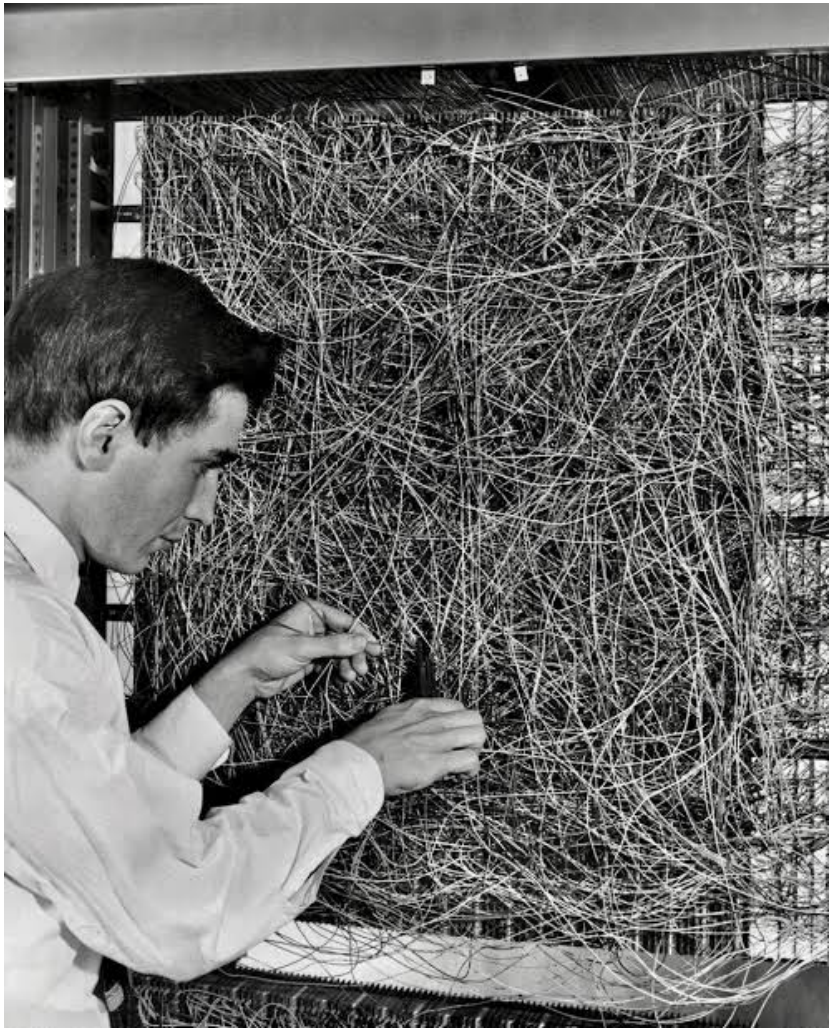
- Use the chain rule to "backpropagate" errors.

The chain rule of differentiation



[1673] Liebniz

Chain rule applied to the perceptron



[1976] Rosenblat

Chain rule for deep neural netowrks



[1982] Werbos

Backpropagation given a name

1986: Learning representations by back-propagating errors.



Rumelhart

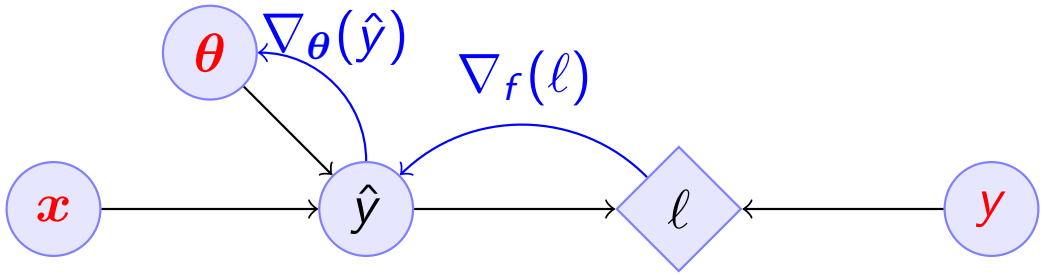


Hinton



Williams

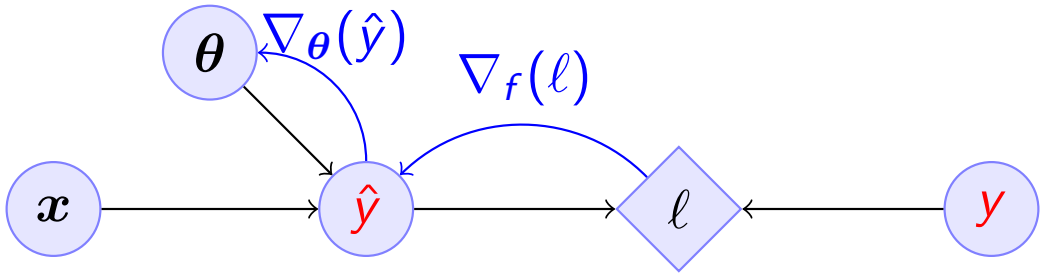
Elementary back-propagation: linear regression



- $f : X \to Y$, $\ell : Y \times Y \to \mathbb{R}$, chain rule: $\nabla_{\boldsymbol{\theta}} \ell = \nabla_{\boldsymbol{\theta}} f \nabla_{\hat{y}} \ell$
- Forward: follow the arrows to calculate variables

$$\hat{y} \triangleq f(\boldsymbol{\theta}, x) = \sum_{i=1}^{n} \theta_i x_i, \qquad \ell(\hat{y}, y) = (\hat{y} - y)^2$$

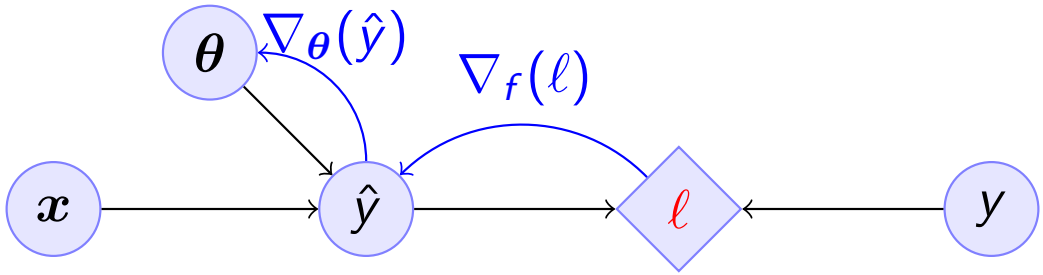
Elementary back-propagation: linear regression



- $f : X \to Y$, $\ell : Y \times Y \to \mathbb{R}$, chain rule: $\nabla_{\boldsymbol{\theta}} \ell = \nabla_{\boldsymbol{\theta}} f \nabla_{\hat{y}} \ell$
- Forward: follow the arrows to calculate variables

$$\hat{y} \triangleq f(\boldsymbol{\theta}, x) = \sum_{i=1}^{n} \theta_i x_i, \qquad \ell(\hat{y}, y) = (\hat{y} - y)^2$$

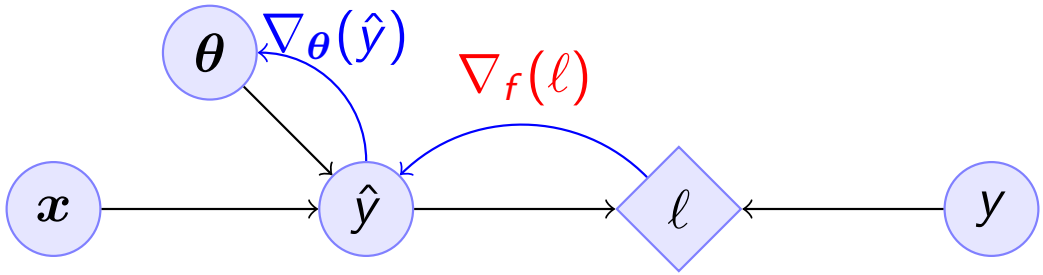
Elementary back-propagation: linear regression



- $f : X \to Y$, $\ell : Y \times Y \to \mathbb{R}$, chain rule: $\nabla_{\boldsymbol{\theta}} \ell = \nabla_{\boldsymbol{\theta}} f \nabla_{\hat{y}} \ell$
- Forward: follow the arrows to calculate variables

$$\hat{y} \triangleq f(\boldsymbol{\theta}, x) = \sum_{i=1}^{n} \theta_i x_i, \qquad \ell(\hat{y}, y) = (\hat{y} - y)^2$$

Elementary back-propagation: linear regression



- $f : X \to Y$, $\ell : Y \times Y \to \mathbb{R}$, chain rule: $\nabla_{\boldsymbol{\theta}} \ell = \nabla_{\boldsymbol{\theta}} f \nabla_{\hat{y}} \ell$
- Forward: follow the arrows to calculate variables

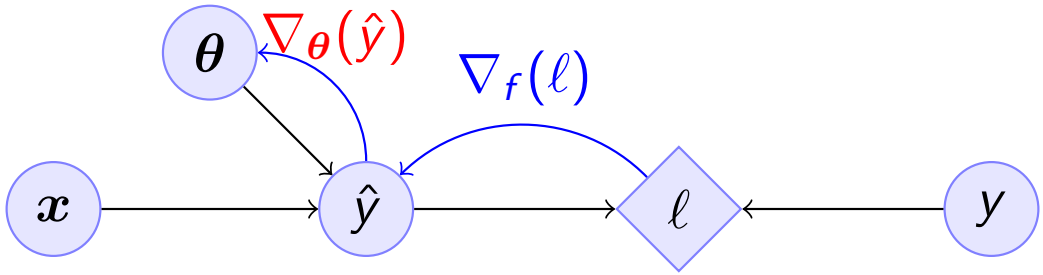
$$\hat{y} \triangleq f(\boldsymbol{\theta}, x) = \sum_{i=1}^{n} \theta_i x_i, \qquad \ell(\hat{y}, y) = (\hat{y} - y)^2$$

- Backward: return to calculate the gradients

$$\nabla_{\boldsymbol{\theta}} \ell(\hat{y}, y) = \nabla_{\boldsymbol{\theta}} f(\boldsymbol{\theta}, \boldsymbol{x}) \times \nabla_{\hat{y}} \ell(\hat{y}, y) \tag{4}$$

$$= \boldsymbol{x} \times 2[\hat{y} - y] \tag{5}$$

Elementary back-propagation: linear regression



- $f : X \to Y$, $\ell : Y \times Y \to \mathbb{R}$, chain rule: $\nabla_{\boldsymbol{\theta}} \ell = \nabla_{\boldsymbol{\theta}} f \nabla_{\hat{y}} \ell$
- Forward: follow the arrows to calculate variables

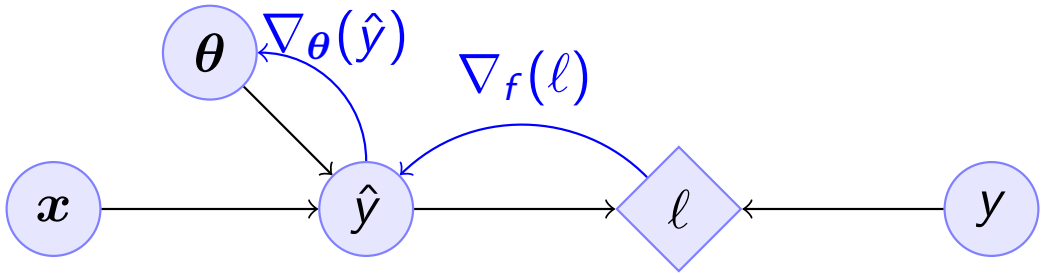
$$\hat{y} \triangleq f(\boldsymbol{\theta}, x) = \sum_{i=1}^{n} \theta_i x_i, \qquad \ell(\hat{y}, y) = (\hat{y} - y)^2$$

- Backward: return to calculate the gradients

$$\nabla_{\boldsymbol{\theta}} \ell(\hat{y}, y) = \nabla_{\boldsymbol{\theta}} f(\boldsymbol{\theta}, \boldsymbol{x}) \times \nabla_{\hat{y}} \ell(\hat{y}, y) \tag{4}$$

$$= \boldsymbol{x} \times 2[\hat{y} - y] \tag{5}$$

Elementary back-propagation: linear regression



- $f : X \to Y$, $\ell : Y \times Y \to \mathbb{R}$, chain rule: $\nabla_{\boldsymbol{\theta}} \ell = \nabla_{\boldsymbol{\theta}} f \nabla_{\hat{y}} \ell$
- Forward: follow the arrows to calculate variables

$$\hat{y} \triangleq f(\boldsymbol{\theta}, x) = \sum_{i=1}^{n} \theta_i x_i, \qquad \ell(\hat{y}, y) = (\hat{y} - y)^2$$

- Backward: return to calculate the gradients

$$\nabla_{\boldsymbol{\theta}} \ell(\hat{y}, y) = \nabla_{\boldsymbol{\theta}} f(\boldsymbol{\theta}, \boldsymbol{x}) \times \nabla_{\hat{y}} \ell(\hat{y}, y) \tag{4}$$
$$= \boldsymbol{x} \times 2[\hat{y} - y] \tag{5}$$

- Update:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \alpha_t \times \nabla_{\boldsymbol{\theta}} \ell(\hat{y}^t, y^t)$$

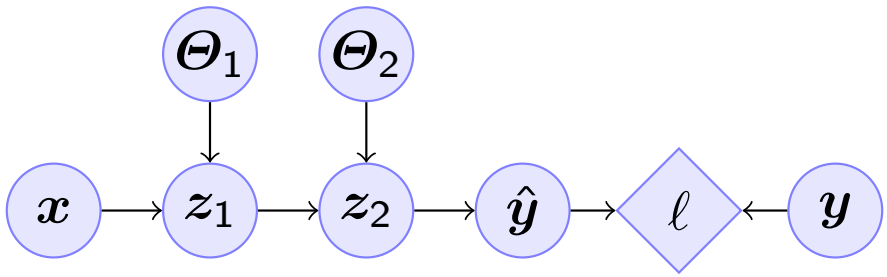
Gradient descent with *back-propagation*

- Dataset D, cost function $L = \sum_t \ell_t$
- Parameters $\boldsymbol{\Theta}_1, \ldots, \boldsymbol{\Theta}_k$ with k layers
- Intermediate variables: $\boldsymbol{z}_j = h_j(\boldsymbol{z}_{j-1}, \boldsymbol{\Theta}_j)$, $\boldsymbol{z}_0 = \boldsymbol{x}$, $\boldsymbol{z}_k = \hat{\boldsymbol{y}}$.

Gradient descent with *back-propagation*

- Dataset D, cost function $L = \sum_t \ell_t$
- Parameters $\boldsymbol{\Theta}_1, \ldots, \boldsymbol{\Theta}_k$ with k layers
- Intermediate variables: $\boldsymbol{z}_j = h_j(\boldsymbol{z}_{j-1}, \boldsymbol{\Theta}_j)$, $\boldsymbol{z}_0 = \boldsymbol{x}$, $\boldsymbol{z}_k = \hat{\boldsymbol{y}}$.

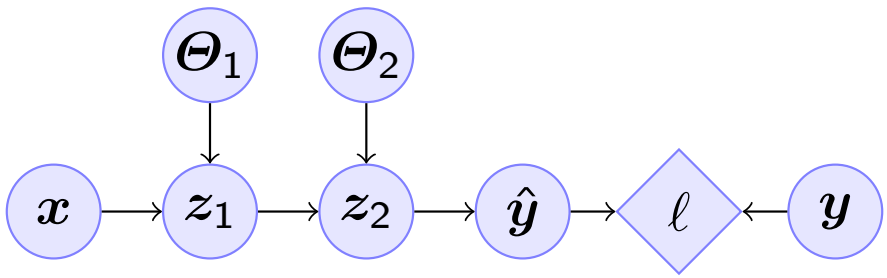
Dependency graph



Gradient descent with *back-propagation*

- Dataset D, cost function $L = \sum_t \ell_t$
- Parameters $\boldsymbol{\Theta}_1, \ldots, \boldsymbol{\Theta}_k$ with k layers
- Intermediate variables: $\boldsymbol{z}_j = h_j(\boldsymbol{z}_{j-1}, \boldsymbol{\Theta}_j)$, $\boldsymbol{z}_0 = \boldsymbol{x}$, $\boldsymbol{z}_k = \hat{\boldsymbol{y}}$.

Dependency graph



Backpropagation with steepest stochastic gradient descent

- Forward step: For $j = 1, \ldots, k$, calculate $\boldsymbol{z}_j = h_j(k)$ and $\ell(\hat{\boldsymbol{y}}, \boldsymbol{y})$
- Backward step: Calculate $\nabla_{\hat{\boldsymbol{y}}} \ell$ and $d_j \triangleq \nabla_{\boldsymbol{\Theta}_j} \ell = \nabla_{\boldsymbol{\Theta}_j} z_j d_{j+1}$ for $j = k \ldots, 1$
- Apply gradient: $\boldsymbol{\Theta}_j -= \alpha d_j$.

Other algorithms and gradients

Natural gradient

Defined for probabilistic models

ADAM

Exponential moving average of gradient and square gradients

BFGS: Broyden–Fletcher–Goldfarb–Shanno algorithm

Newton-like method

Linear layer

Definition

This is a linear combination of inputs $x \in \mathbb{R}^n$ and parameter matrix $\boldsymbol{\Theta} \in \mathbb{R}^{m \times n}$

$$\text{where } \boldsymbol{\Theta} = \begin{bmatrix} \boldsymbol{\theta}_1 \\ \vdots \\ \boldsymbol{\theta}_i \\ \vdots \\ \boldsymbol{\theta}_m \end{bmatrix} = \begin{bmatrix} \theta_{1,1} & \cdots & \theta_{1,j} & \cdots & \theta_{1,m} \\ \vdots & \ddots & \vdots & \ddots & \cdots \\ \theta_{i,1} & \cdots & \theta_{i,j} & \cdots & \theta_{i,m} \\ \vdots & \ddots & \ddots & \ddots & \cdots \\ \theta_{n,1} & \cdots & \theta_{i,j} & \cdots & \theta_{n,m} \end{bmatrix}$$

$$f(\boldsymbol{\Theta}, \boldsymbol{x}) = \boldsymbol{\Theta}\boldsymbol{x} \qquad f_i(\boldsymbol{\Theta}, \boldsymbol{x}) = \boldsymbol{\theta}_i \cdot \boldsymbol{x} = \sum_{j=1}^{n} \theta_{i,j} x_j,$$

Gradient

Each partial derivative is simple:

$$\frac{\partial}{\partial \theta_{i,j}} f_k(\boldsymbol{\Theta}, \boldsymbol{x}) = \sum_{k=1}^{n} \frac{\partial}{\partial \theta_{i,j}} \theta_{i,k} x_k = x_j$$

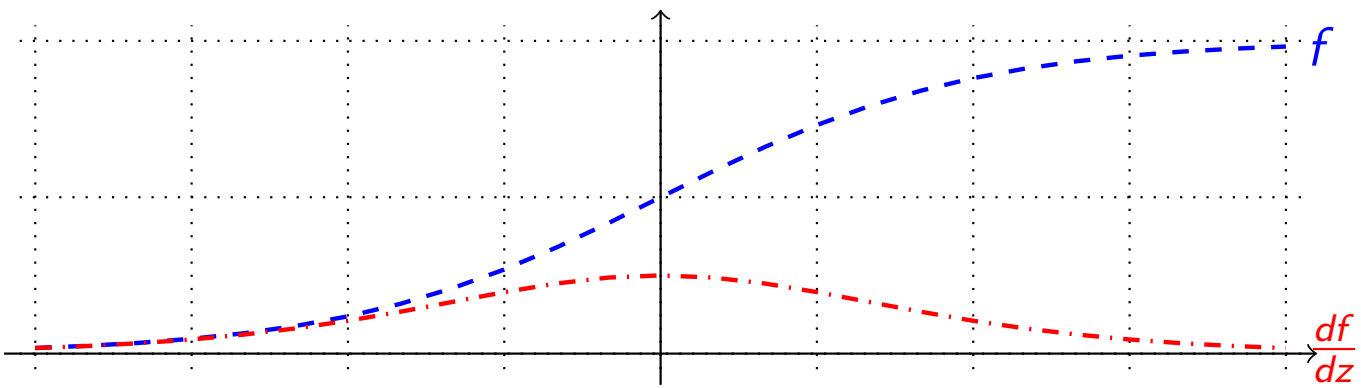
Sigmoid layer

- This layer is used for binary classification.
- It is used in the logistic regression model to obtain label probabilities.

$$f(z) = 1/(1 + \exp(-z))$$

- Derivative

$$\frac{d}{dz} f(z) = \exp(-z)/[1 + \exp(-z)]^2$$



Softmax layer

- This layer is used for multi-class classification
- It is a straightforward generalisation of the sigmoid function.

$$y_i(\boldsymbol{z}) = \frac{\exp(z_i)}{\sum_j \exp(z_j)}$$

Derivative

$$\frac{\partial}{\partial z_i} y_i(\boldsymbol{z}) = \frac{e^{z_i} e^{\sum_{j \neq i} z_j}}{\left(\sum_j e^{z_j}\right)^2}$$

$$\frac{\partial}{\partial z_i} y_k(\boldsymbol{z}) = \frac{e^{z_i + z_k}}{\left(\sum_j e^{z_j}\right)^2}$$

Classification cost functions

Classification error

If z is the output for each class then

$$\ell(z, y) = \mathbb{I}\{y \notin \arg\max(z)\}$$

This is not differentiable.

Error margin

If z is the positive class output then

$$\ell(z, y) = -\mathbb{I}\{zy < 0\} zy$$

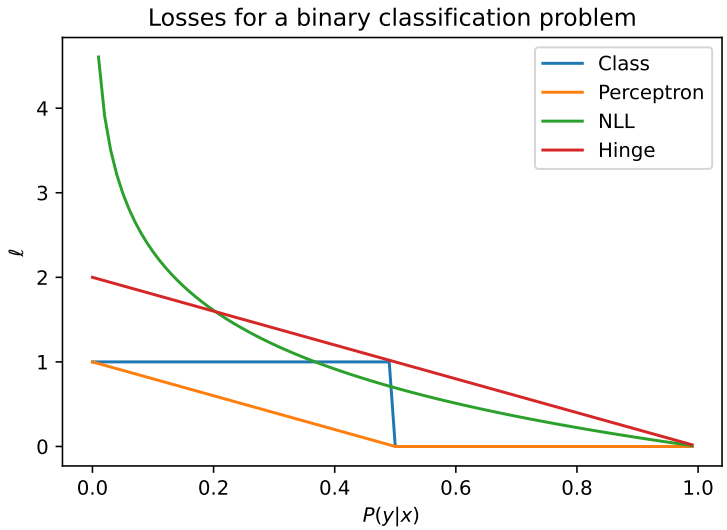
Used in the perceptron.

Negative log likelihood

If z are label probabilities, then

$$\ell(z, y) = -\ln z_y.$$

Used in logistic regression.



Hinge loss

If z are the output for each class

$$\ell(z, y) = 1 - z_y$$

Used in large margin classifiers.

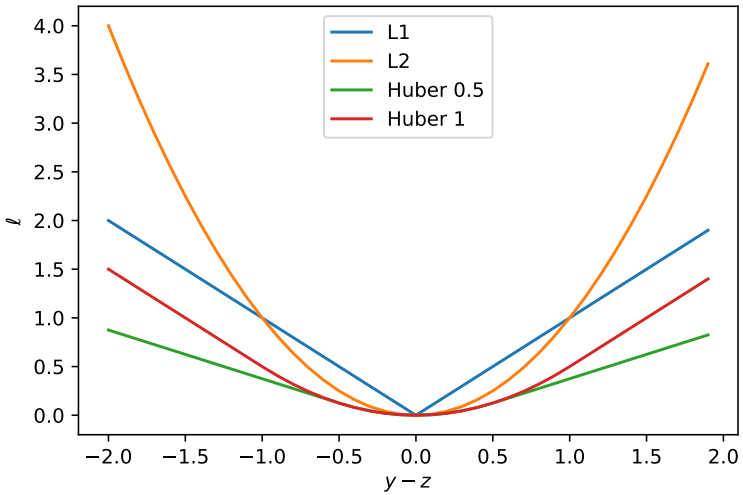
Regression cost functions

L2 loss (Squared error)

If z is a prediction for y then

$$\ell(z, y) = (y - z)^2$$

This is equivalent to negative log likelihood under Gaussianity. Used in linear regression.



L1 loss

If z is a prediction for y then

$$\ell(z, y) = |y - z|$$

Used in LASSO regression.

Huber loss

If z is a prediction, then

$$\ell(z, y) = \begin{cases} \frac{1}{2}(z - y)^2 & |z - y| \leq \delta \\ \delta(|z - y| - \frac{1}{2}\delta) & \text{otherwise.} \end{cases} \quad (6)$$

Mixes L1 and L2 losses.

Continuous and Smooth functions

Definition (Lipschitz continuous functions)

A function $f : \mathbb{R}^n \to \mathbb{R}^m$ is L-Lipschitz continuous with respect to some norm $\|\cdot\|$ if:

$$\|f(x) - f(y)\| \leq L\|x - y\|.$$

Definition (Smooth function)

A function $f : \mathbb{R}^n \to \mathbb{R}^m$ is ℓ-smooth with respect some norm $\|\cdot\|$ if:

$$\|\nabla f(x) - \nabla f(y)\| \leq \ell\|x - y\|.$$

Contraction mappings

A function $f : \mathbb{R}^d \to \mathbb{R}^d$ is a contraction for some norm $\|\cdot\|$ if

$$\|f(x) - f(y)\| \leq c\|x - y\|, \qquad 0 < c < 1.$$

In other words, it is a contraction if it is c-Lipschitz, for $c < 1$. In addition, contraction mappings have a fixed point x^* such that $f(x^*) = x^*$.

- The usual way to apply a contraction mapping is to set:

$$x^{n+1} = f(x^n)$$

- Then $\|x^n - x^*\| \leq c^n \|x^0 - x^*\|$.

Contraction mappings

A function $f : \mathbb{R}^d \to \mathbb{R}^d$ is a contraction for some norm $\|\cdot\|$ if

$$\|f(x) - f(y)\| \leq c\|x - y\|, \qquad 0 < c < 1.$$

In other words, it is a contraction if it is c-Lipschitz, for $c < 1$. In addition, contraction mappings have a fixed point x^* such that $f(x^*) = x^*$.

- The usual way to apply a contraction mapping is to set:
$$x^{n+1} = f(x^n)$$
- Then $\|x^n - x^*\| \leq c^n \|x^0 - x^*\|$.

Examples. Contraction mappings are nice ways to obtain solutions to equations iteratively. Consider finding $\sqrt{y}$. We can rewrite this as the contraction mapping

$$f(x) = (x + y/x)/2.$$

To see this is correct, find the fixed point:

$$\begin{aligned} x &= (x + y/x)/2 \\ 2x &= x + y/x \\ 2x^2 &= x^2 + y \\ x^2 &= y \end{aligned}$$

In fact, it is a general method for obtaining a fixed point equation. If you need to solve $g(x) = 0$, then $f(x) = x + g(x)$ is naturally a fixed point.

An alternative general method is to use the gradient of the function you need solve.

This corresponds to Newton's method.

Gradient descent as a contraction

Suppose $f : \mathbb{R}^d \to \mathbb{R}$ is convex and ℓ-smooth, Then the mapping

$$\psi(x) \triangleq x - \eta \nabla_x f(x)$$

is a contraction as long as $\eta \leq 2/\ell$.
[See Nesterov 04 or Appendix A of Iterative Privacy Amplification for proofs]

Gradient descent in practice

The ideal gradient descent algorithm:

If we could calculte $\nabla_{\boldsymbol{\theta}} \mathbb{E}_{\boldsymbol{\theta}}[L]$, we could do:

$$\boldsymbol{\theta}^{n+1} = \boldsymbol{\theta}^{n} - \alpha_n \nabla_{\boldsymbol{\theta}} \mathbb{E}_{\boldsymbol{\theta}}[L]$$

for a suitable α_n schedule.

Gradient descent on the empirical error

Since we only have the data, we can try to minimse the empirical loss $\frac{1}{T}\sum_{t=1}^{T} \ell(x^t, y^t, \boldsymbol{\theta})$ through gradient descent

$$\boldsymbol{\theta}^{n+1} = \boldsymbol{\theta}^{n} - \alpha_n \frac{1}{T} \sum_{t=1}^{T} \nabla_{\boldsymbol{\theta}} \ell(x^t, y^t, \boldsymbol{\theta})$$

This is also called batch gradient descent.

Stochastic gradient descent

Gradient descent on one example:

We don't have to wait calculate $\nabla_{\boldsymbol{\theta}} \ell(x^t, y^t, \boldsymbol{\theta})$ for all t before applying the update. We can do it at every example:

$$\boldsymbol{\theta}^{n+1} = \boldsymbol{\theta}^n - \alpha_n \nabla_{\boldsymbol{\theta}} \ell(x_{[n]_T}, y_{[n]_T}, \boldsymbol{\theta}).$$

Here $[n]_T$ is 1 + n modulo T to ensure $n \in \{1, \ldots, T\}$.

Minibatch gradient descent

However, it is a bit better to look at K examples at a time before we change the parameters. This is called a minibatch

$$\boldsymbol{\theta}^{n+1} = \boldsymbol{\theta}^n - \alpha_n \frac{1}{K} \sum_{k=nK}^{(n+1)K-1} \nabla_{\boldsymbol{\theta}} \ell(x_{[k]_T}, y_{[k]_T}, \boldsymbol{\theta})$$

This also helps with parallelisation, since we can compute $\ell, \nabla_{\boldsymbol{\theta}} \ell$ in parallel for each example.

Hebbian learning

Let us consider neural connections θ_{ij} between inputs x_i and outputs y_j so that

$$y_j^t = f\left(\sum_k x_k^t \theta_{kj}^t\right),$$

where f is a sigmoid. The principle of Hebbian learning is to increase the strength of those connections whose input contributed to an activation. When $x, y \in [0, 1]$, one interpretation of this principle is this parameter update:

$$\theta_{ij}^{t+1} = \theta_{ij}^t + \alpha x_i^t y_j^t.$$

Of course, this learning rule diverges, so we need to explicitly normalise by dividing with $\|\theta_{\cdot j}\|$, or to subtract a term proportional to θ_{ij} to stabilise.

Features and layers
- Introduction
- Layers
- Activation functions

Algorithms
- Random projection
- Back propagation
- Derivatives
- Cost functions
- Gradient descent
- Stochastic gradient descent in practice
- Other learning rules

Python libraries
- sklearn
- PyTorch
- TensorFlow

sklearn neural networks

Classification

Uses the cross entropy cost

```
from sklearn.neural_network import MLPClassifier
clf = MLPClassifier(hidden_layer_sizes=(5, 2))
clf.fit(X, y)
clf.predict(x^test)
```

- Main condition is layer sizes.

Regression

```
from sklearn.neural_network import MLPRegressor
model = MLPRegressor(hidden_layer_sizes=(5, 2))
```

PyTorch

Data set-up

```
x_train = torch.tensor(x_train, dtype=torch.float32)
train_dataset = TensorDataset(x_train, y_train)
train_loader = DataLoader(train_dataset, batch_size=16, shuffle=True
```

PyTorch: Manual training

Network setup

```
fc1 = nn.Linear(input_size, hidden_size)  # Input to hidden layer
fc2 = nn.Linear(hidden_size, output_size)  # Hidden layer to output
sigmoid = nn.Sigmoid() # some activation function
criterion = nn.BCELoss() #what loss to minimise
optimizer = optim.SGD(model.parameters(), lr=0.001) # how to minimis
```

Training

```
# Manual forward pass.
z1 = fc1(inputs)  # hidden layer 1
a1 = sigmoid(z1)      # Apply activation for hidden
z2 = fc2(a1)      # Linear combination in output layer
outputs = sigmoid(z2)  # Output layer activation
loss = criterion(outputs, labels) # Specify loss
loss.backward() # Backward pass
optimizer.step() # Update weights
```

TensorFlow

This is another library, no need to use this for this course a